fortbridge/cve-2026-108097-research: docs/GADGETS.md

fortbridge/cve-2026-108097-research: docs/GADGETS.md - Author not stated, GitHub.

- Published: date not stated
- Original: <https://github.com/fortbridge/cve-2026-108097-research/blob/1b8509713064e440068b0561eaf4a2a458831b2b/docs/GADGETS.md>
- Preserved from: https://github.com/fortbridge/cve-2026-108097-research/blob/1b8509713064e440068b0561eaf4a2a458831b2b/docs/GADGETS.md (github-api) on 2026-10-21
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

> **Repository reading copy.** Created from GADGETS.md.

GADGETS.md

`fortbridge/cve-2026-108097-research` at `1b8509713064`, path `docs/GADGETS.md`.

Gadget inventory, FieldUiTable `#region_callback` sink (CVE-2026-108097 lab)

18 October 2026. Target: `drupal108097lab-vulnerable-1` (Drupal 11.4.9, PHP 8.4.26 CLI probes, composer autoload of `/opt/drupal/vendor`). Question answered: **if a writer is ever found for a row's `#region_callback`, what existing callables can the attacker actually invoke with exactly one array argument, and what happens?**

Sink recap: `call_user_func_array($row['#region_callback'], [&$row])`, the attacker picks ANY string / `[Class, method]` callable; it receives ONE argument: the row array (by reference). The return value is then used as an array key (`$trees[$region_name]`), which matters for adjudication: **return-value-only channels are barely observable** (a string return becomes a bogus region name, never rendered; an object/array return raises a TypeError/illegal-offset, with one nuance below).

Method (built, not hand-picked)

1. **Enumerate** (`discover_classes.php`, `enumerate_chunk.php`, `enumerate_functions.php`, driver `run_enumerate.sh`): composer classmap + PSR-4 roots (Drupal\Core/Component, Symfony, Twig, Guzzle, PEAR, Drush, psysh, …) + every `Drupal\<module>\` class under `web/core/modules`, `web/modules/contrib` (redirect, redirect_field_ui, webform, webform_composite), `web/modules`, 9,934 classes scanned (9527 reflected OK, 407 not loadable in CLI, mostly missing-parent or `Resources/` data files; full list in `loadfail.jsonl`), plus 1,318 internal + 139 userland (composer `files` autoload, e.g. `twig_*`) functions. Public static, non-abstract methods only; deduped by declaring class. Our own `lab_bridge` fixture was excluded.
2. **Signature filter** (reflection): include when required-params ≤ 1 AND first param type ∈ {untyped, mixed, array, iterable, Traversable} (object-typed excluded, can't conjure objects over HTTP; scalar/callable-typed excluded). 5,715 candidates → **2,109 included** (`candidates.jsonl`).
3. **Dynamic probe** (`probe_batch.php`, driver `run_probes.sh`): each included candidate invoked as `call_user_func_array($candidate, [&$probeRow])` in batched CLI processes (never the running Apache/Drupal; no HTTP; no bootstrap). `$probeRow` = the lab row shape carrying only benign markers: `/tmp/lab-probe/...` paths, literal `whoami`, callable-name strings `print_r`, a serialized-stdClass string. Per candidate recorded: outcome class (ArgumentCountError / TypeError / other throwable / returned / process-died), return summary, ob-captured output + marker hits, and filesystem side effects, the only writable area was `/tmp/lab-probe/scratch` (cwd, `sys_temp_dir`, `TMPDIR`, `session.save_path` all pinned there; before/after scan per candidate). Batch fatalities (exit/die) were isolated and retried one-per-process. Full data: `results.jsonl` (2,109 rows).
4. **Adjudicate**: every survivor (1,239 returned normally) was triaged (`triage.py` → `triage_buckets.json`), and everything with side effects, marker echo, or a dangerous-sounding name was read at source level. Shaped flat-array confirmations for the PEAR file primitives were re-run in scratch. Drupal statics that need the container threw `ContainerNotInitializedException` in CLI (181), those were source-read for what they WOULD do in a real request.

Totals

- Candidates scanned: **5,715** (4,258 statics, 1,318 internal functions, 139 userland functions)
- Invocable with one array arg (signature filter): **2,109**, all probed
- Probed outcomes: 1,239 returned · 311 TypeError · 198 ArgumentCountError · 181 container-not-initialized · 141 other throwables · 5 process-died
- Filesystem side effects observed with the nested lab row: **1** (`System::mktemp`, a file named with the row's `whoami` marker)

- Dangerous after source adjudication: **7 callables**, none reaching code exec through this sink (see ceiling)

Adjudicated gadget table

callable	kind	arg tolerance	effect with array input	exploit value	notes
`Drupal\webform\WebformSubmissionForm::submitFormValues`	static (contrib webform)	`array $values` req=1	**Creates AND saves a webform submission entity from array content** (`WebformSubmission::create($values)` → form validate → `$submission->save()` if no errors), firing webform handlers	**content-injection / DB-write (conditional)**	web/modules/contrib/webform/src/WebformSubmissionForm.php:3312. Needs container (real request OK; probe threw CNI). Conditional: array must carry a valid `webform_id` + pass the webform's validation; handler side effects (emails, post-processing) follow the webform's own config. Source-adjudicated; not executed against the live site.
`System::cat`	static (pear-core-minimal)	untyped req=1	**Reads files listed in the array; with `>` / `>>` elements writes/appends a copy to the named output file** (fopen	**file-write + file-read (conditional)**	vendor/pear/pear-core-minimal/src/System.php:323. Nested lab row → ValueError. Shaped flat row confirmed in scratch: `System::cat(`

callable	kind	arg tolerance	effect with array input	exploit value	notes
			wb/ab+), keyed entirely off array content		`['src','>','dst'])` copied the file. File *contents* only returned, not echoed, through this sink the read half is barely observable; the write half is the payload. URLs work via fopen wrappers per its own docblock (remote-fetch speculation not tested; allow_url_fopen dependent).
`System::rm`	static (pear-core-minimal)	untyped req=1	**Deletes files/dirs named in the array** (`unlink` / `rmdir` per element)	**file-delete (conditional)**	System.php:210. Nested row → TypeError at first nested element; shaped flat row confirmed in scratch: `System::rm(['file'])` deleted it.
`System::mkDir`	static (pear-core-minimal)	untyped req=1	**Creates directories named in the array** (with `-p`, recursively)	**file-write, dirs (conditional)**	System.php:254. Probe: created dir `print_r` in cwd from row content, then TypeError on the next nested

callable	kind	arg tolerance	effect with array input	exploit value	notes
					element (partial side effect before the error). Shaped flat row confirmed: `System::mkDir(['/tmp/lab-probe/scratch/mkdir-proof'])` → created.
`System::mktemp`	static (pear-core-minimal)	untyped req=1	**Creates a temp file (or `-d` dir) whose name prefix and target dir come from array content**; registered shutdown handler deletes it at request end	**file-write, transient (observed)**	System.php:395. The one side effect observed with the actual nested lab row: created `…/whoami<random>` in the probe temp dir. In a real request: file in the system temp dir, empty content, attacker-influenced prefix, auto-removed at shutdown.
`dd`	function (symfony/var-dumper `files`)	variadic mixed	**Dumps the full row into the response (HTML dumper under web SAPI), sends `HTTP/1.1`**	**info-disclosure (row content) + per-request DoS**	vendor/symfony/var-dumper/Resources/functions/dump.php:46. Probe process died by design. Row in a real integration

callable	kind	arg tolerance	effect with array input	exploit value	notes
			`500`, then `exit(1)`		can hold sensitive field values → echoed.
`Symfony\Component\VarDumper\VarDumper::dump`	static	mixed req=1	Dumps the full row (HTML-formatted under web SAPI) into the output; returns the row	info-disclosure (row content)	vendor/symfony/var-dumper/VarDumper.php:41. CLI probe captured nothing (CliDumper writes php://stdout directly); source confirms web output path. Same class of gadget as the already-proven `print_r`.
`print_r` / `var_dump` / `var_export` / `debug_zval_dump`	function (internal)	mixed	Echo full row content	info-disclosure	The known output gadgets; `print_r` already demonstrated end-to-end over HTTP in this lab (`RCE-CAPABILITY-WITNESS.md`).
`Drupal::unsetContainer`	static (core)	no params	Sets `\Drupal::$container = NULL` mid-request → every subsequent `\Drupal::service()` throws → request fails	per-request DoS	web/core/lib/Drupal.php:162. Returned normally in probe.

callable	kind	arg tolerance	effect with array input	exploit value	notes
`Drupal\language\ConfigurableLanguageManager::rebuildServices`	static (core)	no params	`\Drupal::service('kernel')->invalidateContainer()` → next request rebuilds the DI container	DoS-adjacent (cache/invalidate, slow next request)	web/core/modules/language/src/ConfigurableLanguageManager.php:104. Probe threw CNI; source-read.
`Drupal\jsonapi\Routing\Routes::rebuild`	static (core jsonapi)	no params	Invalidates `jsonapi_resource_types` cache tag + marks router rebuild needed	DoS-adjacent (route rebuild on next request)	web/core/modules/jsonapi/src/Routing/Routes.php:476. Probe threw CNI; source-read.
`Symfony\Component\HttpFoundation\Request::setAllowedHttpMethodOverride`	static	array req=1	Replaces the static method-override allowlist with array content	config-mutation, request-scoped, minor	vendor/symfony/http-foundation/Request.php:743. PHP statics reset per request.
`session_set_cookie_params`	function (internal)	array req=1	Sets session cookie lifetime/path/domain/secure/httponly/samesite from array keys for the rest of the request	config-mutation, request-scoped, minor	PHP internal. Returned true in probe.
`Drupal\Component\FileCache\FileCacheFactory::setConfiguration`	static	untyped req=1	Replaces static FileCache configuration (incl. `directory` / `prefix` used for later cache file paths) with array content	config-mutation, request-scoped, conditional	web/core/lib/Drupal/Component/FileCache/FileCacheFactory.php:93. Could redirect FileCache reads/writes for components that construct caches after

callable	kind	arg tolerance	effect with array input	exploit value	notes
					the call, same request only.
`PEAR_ErrorStack::staticPushCallback`	static	untyped req=1	Pushes the raw array onto the global error-callback override stack **without validation**; later `PEAR_ErrorStack::push()` would `call_user_func` it	callback-plant, request-scoped, inert for a non-callable row	vendor/pear/pear-core-minimal/src/PEAR/ErrorStack.php:433. Our row is not a valid callable → later push would just warn. `PEAR::registerShutdownFunc` (PEAR.php:284) is the same pattern at shutdown.
`Composer\InstalledVersions::reload`	static	untyped req=1	`self::$installed = $row`, corrupts composer package metadata for the rest of the request	request-scoped tamper, cosmetic	vendor/composer/InstalledVersions.php:319.
`Drupal\Core\Site\Settings::getAll`	static (core)	no params	Returns ALL settings, hash_salt, DB info	**none through this sink**	web/core/lib/Drupal/Core/Site/Settings.php:127. Probe returned null (no bootstrap); in a real request it returns the secrets array, but the sink only uses the return as an array key (illegal-offset TypeError), so nothing is

callable	kind	arg tolerance	effect with array input	exploit value	notes
					exfiltrated. Kept for honesty: dangerous-looking, unobservable here.
`Symfony\Component\String\LazyString::fromCallable`	static	`callable\	array` req=1	Accepts the row because `count($row) > 2` bypasses its guard; would execute `$row[0]()` as a PHP function **when the LazyString is cast to string**	**none through this sink** (the sink uses the return as an array key → TypeError before any cast)
`Psy\Shell::debug`	static (psysh, via drush)	`array $vars` req=0/1	Enters a psysh debug session with the array as scope	hang/DoS risk, no code exec from content	vendor/psy/psysh/src/Shell.php:355. Probe returned normally (stdin /dev/null). Under mod_php the REPL has no usable input; worst case a hung request.
`Twig\Extension\CoreExtension::captureOutput`	static	`iterable` req=1	Echoes each top-level row element (strings only; nested arrays warn) and returns the capture	info-disclosure, minimal (top-level strings)	vendor/twig/twig/src/Extension/CoreExtension.php:2202.

callable	kind	arg tolerance	effect with array input	exploit value	notes
`System::find`	static (pear)	untyped req=1	Filesystem enumeration keyed off array content; returns matches	read-only recon, return unobservable through sink	System.php:568. Returned normally.
`assert`	function (internal)	mixed	truthiness check on the array	none	PHP 8.4 removed string-code assertions; `assert($row)` just returns true. Named-suspect cleared by PHP version.

Also explicitly checked and cleared: Symfony ExpressionLanguage (no statics invocable, instance `evaluate()` only), PropertyAccess (`createPropertyAccessor()` no-arg factory), Symfony Yaml / Drupal Yaml (`dump`/`encode` return a string rendering; `decode`/`parse` are string-typed → excluded), Twig statics beyond the table (escaping/casting helpers), `SecurityAdvisory::createFromArray` (UnexpectedValueException), entity `::create` statics (`EntityBase`, `ViewUI`, `Action`, `BaseFieldDefinition`, in-memory object construction only; `save()` is an instance method the sink cannot reach), `WebformElementHelper::process` / `Drush\Drupal\DrupalUtil::drushRender` (element rendering; need container; benign transforms), render `preRender*` statics (benign array transforms).

Honest ceiling statement

With one array argument and the exact observable channels this sink offers (echoed output = observable; side effects = observable; return value = used as an array key, i.e. effectively discarded):

- **Worst case found: persistent DB content injection** via `Drupal\webform\WebformSubmissionForm::submitFormValues`, a webform submission created and saved from array content (with that webform's handlers firing), conditional on the array carrying a valid `webform_id` and passing validation. Requires the webform contrib module (present here, not in stock core).
- **File-level primitives** from PEAR `System::cat` / `rm` / `mkDir` / `mktemp`: read+write/append-copy, delete, mkdir, all keyed off array content, all conditional on the row being a flat string list (the nested render-array shape TypeErrors; only `mktemp` fired on the real lab row). No content control beyond file-copy semantics; no path traversal guards, but also no arbitrary bytes written except copies of existing files.

- **Info disclosure**: full row echo (`print_r` family, `VarDumper::dump`, `dd` + request kill).
- **DoS**: `unsetContainer` (per-request), container/router rebuild flags (next-request cost), `dd` (request kill), psysh debug (hang risk).
- **Not found: any code-execution or command-execution gadget reachable with one array argument.** Every shell/`eval`/`include`/unserialize/string-`assert` path needs either a string first argument (TypeErrors on the row), ≥2 required arguments, or an object we cannot build. The single closest call, `LazyString::fromCallable`, defers execution behind a string cast that this sink never performs. This matches and explains the demonstrated exploit shape in this lab: the working RCE path is a callable that *executes row content it selects* (the `execMarkup` bridge pattern), not a bare builtin.

Archived from https://github.com/fortbridge/cve-2026-108097-research/blob/1b8509713064e440068b0561eaf4a2a458831b2b/docs/GADGETS.md. Rendered offline from the local Markdown copy.